Data Reduction: Finishing the DFT Story and Meeting PCA

Big Data Management — Week 5

This lecture closes the loop on the Discrete Fourier Transform (DFT) example started earlier, extracts the general *principle* of data reduction hiding inside it, draws a hard boundary around where that principle should **never** be applied, and then opens the door to a second, fundamentally different reduction technique: **Principal Component Analysis (PCA)**.

Index

- 1. Recap — Where We Left the DFT Example — *the magnitude table nobody explained, finally explained.*
- 2. The Conjugate-Symmetry Shortcut — *how half your Fourier coefficients are freeloaders.*
- 3. Energy Compaction — The Real Idea Behind the Reduction — *throwing away the boring numbers and calling it science.*
- 4. Counting the Storage — From 8 Numbers to 3 — *complex numbers and their sneaky two-for-one storage tax.*
- 5. Reconstruction — Getting (Almost) Your Signal Back — *the inverse DFT, now with amnesia about the numbers you threw away.*
- 6. The Trade-off: Accuracy vs. Storage — *there is no free lunch, only cheaper lunch with fewer calories of accuracy.*
- 7. When NOT to Use Lossy Reduction — The Medical Data Warning — *why your blood pressure should never be "approximately" correct.*
- 8. Enter PCA — The Height & Weight Story — *two variables gossiping about the same thing.*
- 9. What PCA Actually Does — *combining correlated variables into "principal components," a fancier way of saying "one number that does the job of two."*
- 10. Worked Example — Maths & Physics Marks — *four students, one diagonal line, and a 50% storage discount.*
- 11. Practice Questions — *because reading is not the same as understanding.*

---

1. Recap — Where We Left the DFT Example

In the previous lecture, an 8-point signal $x(n)$ was transformed using the Discrete Fourier Transform into 8 frequency-domain coefficients $X(k)$, for $k = 0, 1, \dots, 7$. Since these coefficients are, in general, **complex numbers**, what matters most for understanding their relative importance is their **magnitude**. The table below is the summary carried forward from the last lecture:

Coefficient	Magnitude
$X(0)$	20
$X(1)$	4.18
$X(7)$	4.18
$X(2)$	1.41
$X(6)$	1.41
$X(3)$	0.58
$X(5)$	0.58
$X(4)$	0

The original 8-point time-domain signal was:

$$x(n) = \{2,\ 3,\ 4,\ 4,\ 3,\ 2,\ 1,\ 1\}$$

Notice something interesting: several rows share the exact same magnitude. That is not a coincidence — it comes from a symmetry property that every real-valued DFT obeys.

2. The Conjugate-Symmetry Shortcut

For a real-valued input signal, the DFT coefficients satisfy:

$$X(N-k) = X^*(k)$$

where $X^*(k)$ denotes the **complex conjugate** of $X(k)$, and N is the total number of samples ($N = 8$ in this example, with $k = 0, 1, 2, \ldots, N-1$).

Plugging in a concrete value makes this concrete: take $k = 2$. Then $N - k = 8 - 2 = 6$, so:

$$X(6) = X^*(2)$$

This is exactly why $X(2)$ and $X(6)$ share the same magnitude (1.41) in the table above — they are complex conjugates of each other, differing only in the sign of the imaginary part. The same logic explains why $X(1)$ and $X(7)$ pair up, and why $X(3)$ and $X(5)$ pair up. This isn't specific to this example — **any real-valued signal's DFT will show this mirror symmetry**, and it is this redundancy that we are about to exploit.

> **Note for Review:** Conjugate symmetry is a favorite exam trap. Remember: you only need to explicitly compute *half* of the DFT coefficients (roughly $k = 0$ to $N/2$); the rest are just conjugates, not new information.

3. Energy Compaction — The Real Idea Behind the Reduction

Look again at the magnitude table. The pattern is unmistakable: magnitude starts high (20), stays fairly high for the next pair (4.18), then keeps shrinking — 1.41, 0.58, and finally 0. This shrinking pattern is called **energy compaction**: most of the "signal energy" is packed into just the first few coefficients.

Since magnitude and energy are used interchangeably in this context, the strategy becomes obvious: **keep the large-magnitude coefficients, discard the small ones.** In this example, the decision is to retain only $X(0)$, $X(1)$, and $X(7)$ — the three largest-magnitude coefficients — and throw away $X(2)$, $X(3)$, $X(4)$, $X(5)$, and $X(6)$.

In words: *most of the signal's energy is concentrated in $X(0)$, $X(1)$, and $X(7)$.*

4. Counting the Storage — From 8 Numbers to 3

Here is where the actual data reduction happens. Instead of storing all 8 coefficients $X(0)$ through $X(7)$, only three values are stored:

$$X(0) = 20$$

$$X(1) = -3.41 - i\,2.41$$

$$X(7) = -3.41 + i\,2.41$$

Now count how many *numbers* (not coefficients) this actually requires:

- $X(0) = 20$ is purely real → **1 number**.
- $X(7)$ is complex, so it needs both a real part and an imaginary part → **2 numbers**.
- $X(1)$ is the complex conjugate of $X(7)$ — its real and imaginary parts are already known the moment $X(7)$ is known (just flip the sign of the imaginary part). So $X(1)$ costs **0 additional numbers**.

Total storage requirement: 3 numbers — namely $\{20,\ -3.41,\ -2.41\}$ — down from the original 8 time-domain samples $\{2, 3, 4, 4, 3, 2, 1, 1\}$.

This drop from 8 numbers to 3 numbers **is** data reduction in action.

5. Reconstruction — Getting (Almost) Your Signal Back

To check how good this reduction is, the signal is reconstructed from only the stored coefficients using the **Inverse DFT** formula:

$$\hat{x}(n) = \sum_{k=0}^{N-1} X(k)\, e^{i\,2\pi nk/N}$$

Normally this sum would run over all $N = 8$ values of k. But since only $X(0)$, $X(1)$, and $X(7)$ were stored, the sum is **truncated** — it uses only those coefficients (the "$k = 1$" term automatically covers its conjugate partner $k = 7$ through the symmetry relation). The reconstruction is denoted $\hat{x}(n)$ (with a hat) precisely to signal that it is an **estimate**, not the exact original.

Substituting $n = 0, 1, 2, \ldots, 7$ one at a time into the truncated sum produces the reconstructed vector:

$$\hat{x}(n) = \{2.1,\ 2.9,\ 3.8,\ 4.0,\ 3.1,\ 2.0,\ 1.1,\ 0.9\}$$

Compare this side by side with the true original signal:

Index n	Original $x(n)$	Reconstructed $\hat{x}(n)$
0	2	2.1
1	3	2.9
2	4	3.8
3	4	4.0
4	3	3.1
5	2	2.0
6	1	1.1
7	1	0.9

The reconstruction is close, but not exact — every entry is off by a small amount. That gap is the price paid for compressing 8 numbers into 3.

6. The Trade-off: Accuracy vs. Storage

The storage saving can be quantified directly:

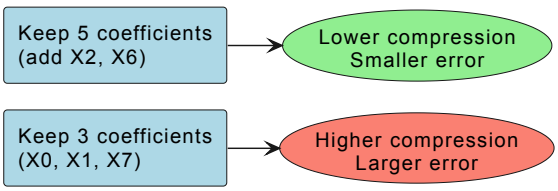
$$\frac{3}{8} \times 100 = 37.5\%$$

So this scheme needs only **37.5%** of the original storage — a **62.5% reduction**. But this reduction is *not free*. The mismatch between $x(n)$ and $\hat{x}(n)$ in the table above is the cost.

This produces a fundamental **trade-off**:

$$\text{Accuracy of Reconstruction} \quad \Longleftrightarrow \quad \text{Storage / Data Reduction}$$

There is no universal rule dictating exactly how many coefficients to keep. Keeping more coefficients (e.g., adding $X(2)$ and $X(6)$, which would cost 2 more numbers since $X(2)$ is complex) reduces the reconstruction error but also reduces the compression ratio — moving from 3 numbers to 5 numbers, for instance, gives a smaller error but a smaller percentage saving. The decision boils down to a practical question: *how much error can be tolerated?* If 10% error is acceptable, one might settle for 3 numbers; if higher fidelity is required, more coefficients must be retained.



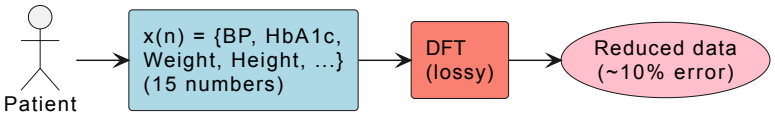
7. When NOT to Use Lossy Reduction — The Medical Data Warning

This is the most important caveat of the whole DFT-based approach: **it is a lossy method**, and lossy methods are dangerous when applied carelessly to sensitive data.

Consider a patient represented as a data object:

$$\text{Patient} \rightarrow x(n) = \{\text{BP},\ \text{HbA1c},\ \text{Weight},\ \text{Height}, \dots\}$$

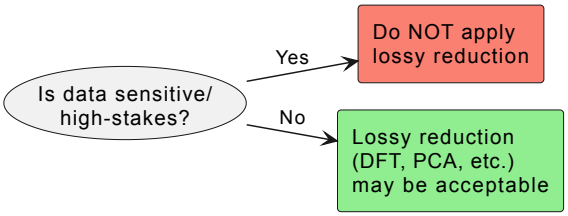
Suppose this vector has 15 numbers describing the patient's vitals and lab values, and the goal is to reduce storage. Applying a DFT here and keeping only the top few coefficients might introduce something like a 10% error.



A 10% error on a blood pressure reading or an HbA1c level is not a harmless rounding artifact — it can distort downstream **medical diagnosis**. Therefore:

> **Rule of thumb:** Do **not** apply DFT — or *any* lossy data reduction method — to sensitive data such as medical records. If storage reduction is genuinely required for such data, only **lossless** methods should be considered (though these typically achieve far lower compression efficiency, which is why they are not the focus of this discussion).

The broader lesson: before reducing any dataset, first **decide whether reduction is even appropriate** for that data, and only then choose a method.



> **Note for Review:** Exam questions often test *judgment*, not just formulas — expect a scenario question asking "should you apply DFT here?" The correct instinct is always: check whether losing information is acceptable for that domain before reaching for a compression technique.

8. Enter PCA — The Height & Weight Story

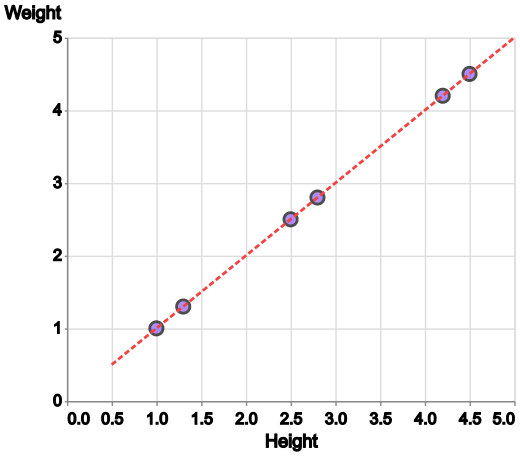
Having seen one method (DFT) that reduces data by exploiting *frequency-domain energy concentration*, the lecture now introduces a second, conceptually different method: **Principal Component Analysis (PCA)**.

$$\text{Principal Component Analysis (PCA)} \longrightarrow \text{Data Reduction}$$

To build intuition, consider a toy dataset of six people, each described by two (deliberately vague, adjective-based) variables — height and weight:

Person	Height	Weight
A	Short	Light
B	Short	Light
C	Medium	Medium
D	Medium	Medium
E	Tall	Heavy
F	Tall	Heavy

If this data were plotted with height on one axis and weight on the other, a clear pattern emerges: as height increases, weight increases too. All six people line up roughly along a **diagonal direction** in this 2D space.



Since height and weight move together — they are **correlated** — there is redundancy in keeping them as two separate variables. Physically, it makes sense to combine them into a single derived quantity, something like an **"overall body size"** score. Instead of storing two numbers per person, only one is needed.

That diagonal direction along which the points line up is not just a visual curiosity — it is the most important idea in PCA:

> **PCA essentially tries to find the direction along which the data varies (the direction of maximum variance).**

The reasoning: moving from person A to person B to person C, *something* is changing — that change is what statisticians call **variance**. The direction along which this change is largest is the direction that best distinguishes the data points from one another. Find that direction, and you have found the most informative "summary axis" of the data.

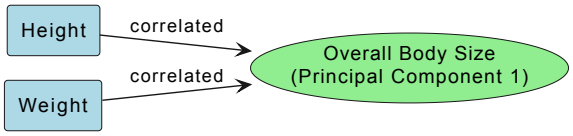
---

9. What PCA Actually Does

Putting it precisely:

> **PCA combines *correlated* variables into new variables called principal components.**

In the height/weight example, the two original variables (height, weight) are combined into one new variable (overall body size) — this new variable is called a **principal component**. In general, a dataset with many variables might be reducible to a smaller number of principal components: for example, 5 original variables might be reduced to just 3 principal components.



The critical pre-condition: **PCA only works if variables are correlated.** If two variables move independently of each other (uncorrelated), there is no redundancy to exploit, and PCA will not usefully compress them.

---

10. Worked Example — Maths & Physics Marks

To make the mechanics concrete, consider four students (A, B, C, D) with (deliberately extreme) marks in Maths and Physics:

Student	Maths	Physics
A	2	2
B	3	3
C	4	4
D	5	5

This data is intentionally trivial — Maths and Physics marks are *identical* for every student, i.e., perfectly correlated — so the PCA procedure and its outcome are easy to verify by hand.

Step 1 — Collect the Data Points

Treat each student as a 2D data point:

$$(2,2),\ (3,3),\ (4,4),\ (5,5)$$

Step 2 — Mean Computation

Compute the mean of each variable across all students:

$$\text{Mean(Maths)} = \frac{2+3+4+5}{4} = 3.5$$

$$\text{Mean(Physics)} = \frac{2+3+4+5}{4} = 3.5$$

(In general the two means need not be equal — they coincide here only because the data was chosen to be perfectly symmetric.)

Step 3 — Mean Centering

Subtract the corresponding mean from every data point. This step is **not optional** — skipping it gives incorrect results.

Student	Original (x, y)	Mean-Centered
A	(2, 2)	$(2-3.5,\ 2-3.5) = (-1.5, -1.5)$
B	(3, 3)	$(3-3.5,\ 3-3.5) = (-0.5, -0.5)$
C	(4, 4)	$(4-3.5,\ 4-3.5) = (0.5, 0.5)$
D	(5, 5)	$(5-3.5,\ 5-3.5) = (1.5, 1.5)$

Step 4a — Find the Direction of Maximum Variance

The central question PCA asks is:

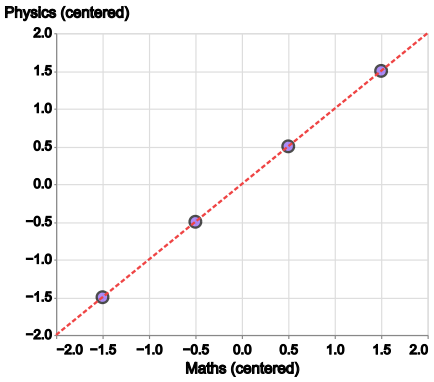
> **Along which direction does the centered data spread/vary the most?**

Plotting $(-1.5, -1.5)$, $(-0.5, -0.5)$, $(0.5, 0.5)$, $(1.5, 1.5)$ shows they all fall exactly on the **diagonal line** $y = x$. (In real datasets this direction is almost never visually obvious — it must be computed mathematically, typically via eigenvectors of the covariance matrix — but here it can be read off directly, which is exactly why this toy example is useful for building intuition.)

A **unit vector** (magnitude 1) is defined along this diagonal direction and called the **first principal component**, $PC1$:

$$PC1 = \frac{1}{\sqrt{2}}\begin{pmatrix}1\\1\end{pmatrix}$$

The $\frac{1}{\sqrt{2}}$ factor normalizes the vector so its length is exactly 1: $\left(\frac{1}{\sqrt{2}}\right)^2 + \left(\frac{1}{\sqrt{2}}\right)^2 = \frac{1}{2} + \frac{1}{2} = 1$.



Step 4b — Project the Centered Data onto PC1

"Projecting" onto $PC1$ means taking the **dot product** of each centered data point with the $PC1$ vector — this collapses each 2D point into a single number, the **PC1 score**.

For student A, with centered point $(-1.5, -1.5)$:

$$\text{PC1 score}_A = \frac{1}{\sqrt{2}}\begin{pmatrix}1\\1\end{pmatrix} \cdot \begin{pmatrix}-1.5\\-1.5\end{pmatrix} = \frac{1}{\sqrt{2}}[(1)(-1.5) + (1)(-1.5)] = \frac{-3}{\sqrt{2}} \approx -2.12$$

Repeating this dot product for every student gives:

Student	Original Data	Mean-Centered	PC1 Score
A	(2, 2)	(-1.5, -1.5)	**-2.12**
B	(3, 3)	(-0.5, -0.5)	**-0.71**
C	(4, 4)	(0.5, 0.5)	**0.71**
D	(5, 5)	(1.5, 1.5)	**2.12**

The Result: 10 Numbers → 5 Numbers

Originally, each of the 4 students required **2 numbers** (Maths mark, Physics mark) — **8 numbers total**, or 10 if you count the fact that in general each variable also needs its own stored mean value for reconstruction later (2 means + 8 data values = 10). PCA replaces the two correlated variables with a **single PC1 score per student**:

$$\text{Total numbers required (with PC1)} = 4 \text{ scores} + 1 \text{ direction vector's worth of bookkeeping} \approx 5 \text{ numbers}$$

That is a **50% reduction** — and, just as with the DFT example, this compression is not free: reconstructing the original marks from the PC1 score alone would reintroduce some error, exactly the same accuracy-vs-storage trade-off seen earlier.

The scores also carry meaning: student D's score (+2.12) is higher than student A's (-2.12), correctly reflecting that D scored better in both subjects than A — the single PC1 number still preserves the *ranking* information of the original two variables.

> **Note for Review:** The reason this example works so cleanly is that Maths and Physics marks are *perfectly* correlated (identical, in fact). In real data, correlation is rarely perfect, so the "diagonal direction" won't be exactly 45°, and it must be found using eigen-decomposition of the covariance matrix rather than by eyeballing a scatter plot. The takeaway concept — project onto the direction of maximum variance — remains exactly the same.

DFT vs. PCA — A Quick Contrast

Aspect	DFT	PCA
Domain of operation	Time → Frequency (uses complex exponentials)	Original variable space → New "component" space
What it exploits	Energy concentration in a few frequency coefficients	Correlation between variables
Core mathematical tool	Complex exponentials, conjugate symmetry	Mean centering, direction of maximum variance (later: eigenvectors)
Requires correlated input?	No	**Yes** — without correlation, PCA gives no benefit

---

11. Practice Questions

1. Suppose a new 8-point signal has DFT magnitudes that are **all exactly equal** (no energy compaction at all). What would this imply about the effectiveness of discarding "small" coefficients, and would keeping just 3 coefficients still be a reasonable strategy? Justify your reasoning.
2. In the medical data example, imagine that instead of blood pressure and HbA1c, the 15 numbers included things like "patient ID" and "zip code." Would your earlier objection to using DFT still apply to *these* particular variables? What distinguishes data where lossy compression is dangerous from data where it might be tolerable?
3. If a dataset has three variables — two of which are strongly correlated and one of which is completely independent of the other two — what would you expect PCA to do with the independent variable? Would you expect to gain any storage reduction from it, and why?
4. In the Maths/Physics example, the reduction went from 2 variables to 1 principal component. If a third subject, say Chemistry, were added with marks that are also perfectly identical to Maths and Physics for every student, how many principal components would you expect to be needed to fully capture the data, and why?
5. The lecture states that DFT exploits energy compaction in the *frequency* domain, while PCA exploits correlation in the *variable* domain. Construct a short argument for why a dataset could respond well to PCA but poorly to a DFT-based reduction (or vice versa) — think about what kind of "structure" each method is actually looking for.